



Research Propos***hmas Taj v3.pdf



Tehmas_25meit00***pdate final.pdf

Reconstruct my entire thesis project from your memory and the attached documents. Create a comprehensive Project Master Document that captures everything about the research, current progress, technical details, datasets, methodology, objectives, and completed work. This document will serve as the single source of truth for the remainder of my thesis.



Gemini 3.1 Pro Extended

Here is the comprehensive Project Master Document reconstructed from the provided materials and your established context. This document synthesizes your methodology, technical requirements, and strategic objectives to serve as the single source of truth for the remainder of your MS-IT research.

Master Thesis Project Document

Title: A User-Centric Business Intelligence Framework for Predictive Customer Churn in E-Commerce

Researcher: Muhammad Tehmas (25MEIT003) **Institution:** Mehran University of Engineering and Technology (MUET) **Supervisor:** Prof. Dr. Shahnawaz Talpur **Co-Supervisor:** Engr. Madeha Memon

Current Phase: Training & Testing / Results & Evaluation (June 2026)

1. Executive Summary & Problem Statement

Customer churn, defined as users discontinuing their engagement or purchases, is a critical challenge in e-commerce, where increasing customer retention by 5% can boost profits by 25% to 95%. In Pakistan's rapidly growing e-commerce sector, high churn is driven by delayed deliveries, lack of trust, and insufficient personalization.

Currently, existing churn prediction strategies are heavily model-driven and operate as "black boxes". They prioritize technical accuracy but lack the interpretability required for decision-makers. This

creates a gap for non-technical SME managers who need actionable, user-centric insights rather than complex mathematical outputs.

Research Aim: To design a user-centric Business Intelligence (BI) framework for e-commerce that accurately predicts customer churn while uncovering the behavioral, experiential, and contextual causes behind it. The framework translates predictive analytics into an actionable format for non-technical SME managers to support data-driven retention strategies.

2. Research Objectives

1. To gather and prepare large quantities of e-commerce customer data encompassing transactional, behavioral, and engagement metrics.
 2. To develop customer churn predictive models integrating Contrastive Learning and Explainable Boosting Machines (EBM).
 3. To examine the specific contextual, behavioral, and experiential variables influencing customer drop-off.
 4. To design a user-friendly BI dashboard that provides actionable, intuitive insights for SME managers.
 5. To validate predictive performance and business impact using both baseline academic datasets and real-world local e-commerce data.
-

3. Methodology: Design Science Research (DSR)

The project utilizes the Design Science Research (DSR) methodology to systematically build and validate a practical IT artifact.

DSR Workflow

- **Step 1: Data Preprocessing:** Cleaning demographics, browsing logs, and transaction histories. Missing values will be imputed (mean for numerical, mode for categorical), continuous features normalized (Min-Max/Z-score), and data augmented to create positive (similar behavior) and negative (dissimilar behavior) pairs.
- **Step 2: Representation Learning:** Utilizing self-supervised Contrastive Learning (SimCLR/BYOL) to generate dense customer embeddings. This step extracts hidden patterns in behavior-engagement without needing fully labeled data.
- **Step 3: Predictive Modeling:** Applying Explainable Boosting Machines (EBM) to learn the relationship between the extracted customer embeddings and churn propensity. EBMs provide inherently transparent predictions by utilizing shallow decision trees in an additive fashion.

- **Step 4: Comparative Analysis:** Benchmarking the hybrid model against traditional and state-of-the-art algorithms, including Random Forest, XGBoost, and LSTM. Evaluation metrics include Accuracy, F1-score, ROC-AUC, calibration, and interpretability.
- **Step 5: BI Integration:** Deploying the feature contributions, risk scores, and probabilities into a user-centric dashboard.

4. Dataset & Validation Strategy

The testing protocol ensures the framework is both theoretically sound and practically applicable in the local market.

Phase 1: Baseline Academic Validation

- **Source:** E-Commerce Customer Churn Dataset (published on Kaggle by samuelsemaya, 2025).
- **Behavioral Attributes:** Tenure, DaySinceLastOrder, OrderCount, CashbackAmount, OrderAmountHike_LastYear, WarehouseToHome, App_surfing_inHrs, Registered_Devices.
- **Categorical Attributes:** PreferredLoginDevice, CityTier, Payment_Method, Gender, Satisfaction_Score, MaritalStatus, Complain, Coupons_Used.
- **Target Variable:** Churn (Binary indicator: 1 = churned, 0 = retained).

Phase 2: Real-World Local Validation

- **Source:** Raw local e-commerce logistics and sales records to stress-test the DSR framework within the Pakistani market context.
- **Application:** Transforming delivery statuses and transaction amounts into RFM (Recency, Frequency, Monetary) metrics to test the real-world operational impact. The real-world transactional and operational data generated by Urban Daraz provides the ideal, immediate local context for this practical validation phase.

5. Technical Stack

The selected tools align perfectly with a "code-light" approach, relying on powerful, high-level libraries and advanced UI design platforms.

Category	Tool / Technology	Application
Primary Language	Python	Core data processing and model orchestration.

Category	Tool / Technology	Application
Deep Learning	PyTorch	Framework for Contrastive Learning (SimCLR/BYOL).
Data Manipulation	Pandas	Handling and transforming large datasets.
Predictive Modeling	Interpret ML	Building and visualizing the Explainable Boosting Machines (EBMs).
Prototyping	Figma	Collaborative UI/UX interface design. Creating a minimalistic, clean aesthetic here ensures the final artifact is truly usable for non-technical users.
Business Intelligence	Power BI	Final deployment of interactive data visualizations and analytics.

6. Project Timeline & Current Status

The project follows a defined timeline from March to August 2026.

- **March - May 2026 (Completed):** Topic Survey, Literature Review, Dataset Preparation, and Churn Model Development.
- **June - July 2026 (Active):** Training & Testing and Results & Evaluation.
- **June - August 2026 (Active/Upcoming):** Thesis & Paper Write-up. Prepare to document all evaluation metrics and synthesize the findings for review by Sir Shahnawaz.

2026-06-28 06:17:53 You Asked 

Act as my thesis supervisor and a Senior Machine Learning researcher. Critically review the Project Master Document. Identify all weaknesses, missing components, research gaps, methodological issues, missing experiments, missing literature, implementation gaps, evaluation gaps, and writing deficiencies. Prioritize them by importance and recommend improvements.

This is a strong foundational document, Tehmas. You have clearly articulated the business value, the methodology, and the timeline. However, stepping into my role as your supervisor and a senior ML researcher, I need to point out a few critical architectural and methodological flaws that will derail your thesis if we don't address them now.

When presenting this at a master's level, the defense committee will scrutinize the technical logic of your pipeline. Right now, there is a fundamental contradiction between your chosen models and your core objective.

Here is my critical review, prioritized from show-stopping structural issues to evaluation gaps.

1. Critical Priority: The Interpretability Paradox (Implementation Flaw)

The most glaring issue in the Project Master Document is the proposed pipeline: **Raw Data** → **Contrastive Learning (SimCLR/BYOL)** → **EBMs** → **Interpretability**.

- **The Flaw:** Explainable Boosting Machines (EBMs) are intrinsically interpretable because they map original, raw features (e.g., *Tenure*, *OrderCount*) directly to the target outcome. If you pass your data through a deep Contrastive Learning model first, the output is a dense, high-dimensional latent vector (e.g., $z_1, z_2, \dots, z_{128}$). If you train your EBM on these embeddings, the EBM will tell you "Embedding dimension z_{42} is the most important factor for churn."
- **The Impact:** An SME manager cannot take action on "Embedding z_{42} ." You will completely lose the behavioral and experiential insights you promised, violating your primary research objective.
- **Recommendation:** Decouple these models.
 - *Path A:* Train the EBM directly on the preprocessed tabular data to extract actionable, feature-level insights for your BI dashboard.
 - *Path B:* Use Contrastive Learning purely for complex customer segmentation or clustering, not as a feature extractor for the EBM.

2. High Priority: Contrastive Learning on Tabular Data (Methodological Gap)

You have proposed using SimCLR or BYOL, which are heavily optimized for Computer Vision (using image cropping, blurring, etc.).

- **The Flaw:** How do you create "positive pairs" and "negative pairs" from a static row in an Excel sheet or SQL database? Tabular data augmentation is notoriously difficult because slightly altering a numerical value (e.g., changing *Satisfaction_Score* from 5 to 4) might change the fundamental meaning of the data point, creating a false positive pair.
- **Recommendation:** You must specify a tabular-specific contrastive learning framework. Look into **SCARF** (Self-Supervised Contrastive Learning using Random Feature Corruption) or **VIME** (Value Imputation and Mask Estimation). Alternatively, since your strength lies more in UI/UX and BI

implementation, consider dropping the deep learning representation phase entirely. EBMs are powerful enough on their own for tabular data and align better with a manageable, reliable technical stack.

● 3. Medium Priority: Missing BI Evaluation Metrics (Evaluation Gap)

Your Comparative Analysis section focuses entirely on ML metrics (Accuracy, F1-score, ROC-AUC).

- **The Flaw:** You are utilizing the Design Science Research (DSR) methodology to build an IT artifact (the dashboard). If you only measure ML accuracy, you are ignoring the "User-Centric" half of your thesis title.
- **The Missing Literature:** In your own literature review, Amirasari and Andrias demonstrated the necessity of checking the feasibility of a BI dashboard in real life using the Technology Acceptance Model (TAM). Omol et al. also emphasized the need to ensure mathematical models are easy to understand for non-technical users.
- **Recommendation:** Introduce a quantitative and qualitative evaluation for the Power BI dashboard. You must measure usability (e.g., System Usability Scale - SUS) and conduct a brief TAM survey with SME managers (or simulated users) to prove that the dashboard actually aids decision-making.

● 4. Low Priority: Data Pipeline Specifics (Writing Deficiency)

Phase 2 of your validation strategy involves "Raw local e-commerce logistics and sales records."

- **The Flaw:** The document lacks specificity regarding the nature of this data. The leap from Kaggle data to raw operational logistics is massive.
- **Recommendation:** Explicitly document the schema mapping. How will you transform delivery dispatches, return statuses, and customer inquiries into the structured behavioral formats (like *App_surfing_inHrs* or *Satisfaction_Score*) that your Phase 1 model expects? State the ETL (Extract, Transform, Load) process clearly.

Next Steps for Revision

To fix the architecture while playing to your strengths in design and high-level logic, I strongly suggest we simplify the backend to amplify the frontend. An interpretable, perfectly designed EBM integrated seamlessly into a beautifully engineered Power BI dashboard is worth much more than a broken Deep Learning pipeline.

How would you prefer to resolve the EBM/Contrastive Learning conflict—should we pivot to a purely EBM-driven approach to guarantee feature interpretability, or do you want to attempt parallel modeling?

Based on the finalized Project Master Document, create a complete roadmap for completing my thesis. Break it into phases with objectives, tasks, deliverables, estimated durations, dependencies, milestones, risks, and weekly action plans. Include a progress dashboard and a realistic schedule from today until thesis submission.

 Gemini 3.1 Pro Extended

Here is your comprehensive thesis completion roadmap. As your supervisor, I have structured this to ensure we hit your August 2026 deadline while decisively correcting the architecture flaw we discussed (decoupling the Deep Learning pipeline from the EBMs).

By streamlining the backend to rely purely on Explainable Boosting Machines (EBMs) for tabular data, we are playing directly to your strengths as a "code-light" UI/UX specialist. This allows us to dedicate more time to crafting a beautiful, highly functional Power BI dashboard that delivers real business value.

 Thesis Progress Dashboard (Current Status: ~45% Complete)

Project Component	Status	Target Completion
Topic Survey & Literature Review	 Completed	March - May 2026
Dataset Preparation (Phase 1)	 Completed	April - May 2026
Model Architecture Redesign	 In Progress	Week 1 (Early July)
Churn Model Development (EBM)	 In Progress	Week 2 (Mid July)
BI Dashboard UI/UX Design (Figma/Power BI)	 Not Started	Week 3-4 (Late July)
Two-Phase Validation (Kaggle + Urban Daraz)	 Not Started	Week 5-6 (Early Aug)
Results, Usability Eval & Thesis Write-up	 In Progress	Week 7-9 (Late Aug)

 Phased Thesis Roadmap (July - August 2026)

Phase 1: Architecture Revision & Core Modeling (Weeks 1-2)

- **Objective:** Correct the pipeline by removing the Contrastive Learning bottleneck and training the Explainable Boosting Machine (EBM) directly on your tabular data to guarantee absolute interpretability.
- **Tasks:**
 - Strip out SimCLR/BYOL from the codebase.
 - Load the preprocessed Kaggle Phase 1 dataset into Python (Pandas).
 - Train the EBM using the `InterpretML` library.
 - Extract global feature importance (overall churn drivers) and local explanations (individual customer risk scores) into structured CSV files for export.
- **Deliverable:** A trained, highly accurate EBM model outputting clean CSV files containing feature contributions and risk probabilities.
- **Dependencies:** Preprocessed Kaggle dataset.

Phase 2: BI Dashboard Design & Integration (Weeks 3-4)

- **Objective:** Build the "User-Centric" core of your thesis. Connect the EBM outputs to Power BI and design an interface tailored for non-technical SME managers.
- **Tasks:**
 - Draft a clean, minimalistic UI layout in Figma (playing to your graphic design strengths).
 - Import the EBM CSV outputs into Power BI.
 - Build interactive visualizations (e.g., At-Risk Customer Lists, Feature Impact Waterfall Charts, RFM segmentations).
- **Deliverable:** A fully functional, aesthetically clean Power BI dashboard displaying actionable predictive insights.
- **Dependencies:** Exported CSVs from the Phase 1 model.

Phase 3: Two-Phase Validation (Weeks 5-6)

- **Objective:** Prove your framework works mathematically and practically in a real-world setting.
- **Tasks:**
 - **Phase 1 (Academic):** Evaluate the Kaggle model against standard ML metrics (Accuracy, ROC-AUC) and compare it against a baseline Random Forest model.
 - **Phase 2 (Local):** Process raw logistics/sales records from your child e-commerce brand (Urban Daraz). Map these to RFM (Recency, Frequency, Monetary) metrics, run them through the EBM, and load them into the dashboard.
 - **Usability Testing:** Conduct a brief System Usability Scale (SUS) or Technology Acceptance Model (TAM) evaluation of the dashboard.

- **Deliverable:** Complete ML performance metrics and a validated, real-world deployment on local SME data.
- **Dependencies:** Functioning Power BI dashboard and Urban Daraz operational data.

Phase 4: Synthesis & Thesis Finalization (Weeks 7-9)

- **Objective:** Document the revised methodology, present the dashboard as your primary IT artifact, and finalize the manuscript for defense.
 - **Tasks:**
 - Update the "Methodology" section of your thesis to reflect the pure EBM + BI approach (Design Science Research).
 - Write the "Results & Evaluation" chapter, emphasizing both model accuracy and BI usability.
 - Format the thesis according to MUET guidelines.
 - **Deliverable:** Final Master's Thesis Document ready for submission to Sir Shahnawaz and Engr. Madeha Memon.
-



Weekly Action Plan (Starting Today: June 28, 2026)

- **Week 1 (June 28 - July 4):** Finalize the EBM script in Python. Ensure it trains properly on the Kaggle data and can export `terms_scores_` and local predictions to CSV.
 - **Week 2 (July 5 - 11):** Run the comparative analysis (EBM vs. Random Forest/XGBoost). Document the accuracy and ROC-AUC scores.
 - **Week 3 (July 12 - 18):** Wireframe the dashboard in Figma. Prioritize a minimal, modern aesthetic. Set up the Power BI workspace and load the CSVs.
 - **Week 4 (July 19 - 25):** Build the actual visualizations in Power BI. Ensure a non-technical manager can look at it and immediately know *who* is churning and *why*.
 - **Week 5 (July 26 - Aug 1):** Execute the Phase 2 local validation. Extract the June/July logistics data from Urban Daraz, format it into RFM metrics, and run it through the system.
 - **Week 6 (Aug 2 - 8):** Conduct the TAM/SUS evaluation. Have a few peers or colleagues act as "SME Managers" to navigate the dashboard and provide structured feedback.
 - **Week 7 (Aug 9 - 15):** Write the Results and Evaluation chapters. Embed screenshots of your polished Power BI dashboard.
 - **Week 8 (Aug 16 - 22):** Final review of the entire thesis draft. Ensure all IEEE citations match the updated literature.
 - **Week 9 (Aug 23 - 30):** Submit the final draft to your supervisors and prepare your defense presentation.
-

⚠ Risk Management

- **Risk:** The Urban Daraz data is too messy or sparse to generate meaningful predictions.
 - **Mitigation:** Fall back on purely descriptive analytics (RFM clustering) for Phase 2, explicitly stating in the thesis that the framework lays the groundwork for predictive capabilities once the local data matures.
- **Risk:** Power BI struggles with dynamic Python integration.
 - **Mitigation:** Stick to the CSV export/import method. It is highly stable, requires no complex API coding, and perfectly satisfies the DSR requirement of building a functional prototype.

This schedule gives us a comfortable buffer for final edits in late August. Before we start coding the EBM export script in Week 1, how much raw operational data (e.g., number of transactions, customer records) do we currently have from the Urban Daraz launch to use for the Phase 2 validation?

2026-06-28 06:23:52 You Asked 

Using the finalized project documentation, act as a Senior ML Engineer. Design the complete technical implementation for the thesis, including dataset preparation, EDA, preprocessing, feature engineering, model development, hyperparameter tuning, evaluation, explainability, Business Intelligence dashboard, and final architecture. Recommend Python libraries, project structure, and best practices.

💠 Gemini 3.1 Pro Extended

This is exactly where the rubber meets the road. Dropping the Deep Learning overhead to focus on a pure Explainable Boosting Machine (EBM) integrated with a high-end Power BI dashboard was the right architectural move. It guarantees the interpretability required by your Design Science Research (DSR) methodology and allows you to lean heavily into your UI/UX and system design strengths.

As a Senior ML Engineer, I will lay out the complete technical blueprint for your thesis. This implementation is designed to be highly modular, allowing you to manage the Python backend comfortably while dedicating your creative energy to the Business Intelligence front end.

📁 1. Project Architecture & Structure

Keep your workspace clean. A well-organized directory prevents chaos when you transition from Phase 1 (Kaggle data) to Phase 2 (Urban Daraz data). Set up your repository like this:

text

```
ecommerce_churn_thesis/
|
|—— data/
|   |—— raw/           # Original Kaggle & Urban Daraz datasets
|   |—— processed/     # Cleaned data ready for modeling
|   |—— outputs/       # Exported EBM scores & risk profiles for Power BI
|
|—— notebooks/        # Jupyter Notebooks for exploration
|   |—— 01_eda_and_profiling.ipynb
|   |—— 02_feature_engineering.ipynb
|
|—— src/              # Modular Python scripts
|   |—— preprocess.py  # Data cleaning logic
|   |—— train_ebm.py   # EBM training and hyperparameter tuning
|   |—— evaluate.py    # ML metrics (ROC-AUC, F1)
|   |—— export_bi.py   # Script to generate flat files for the dashboard
|
|—— requirements.txt   # Python dependencies
|—— README.md
```

Recommended Core Libraries:

- **Data Handling:** pandas , numpy
- **Automated EDA:** ydata-profiling (generates complete HTML reports in one line of code)
- **Modeling & Explainability:** interpret (Microsoft's InterpretML library for EBMs)
- **Evaluation:** scikit-learn (for data splitting, baseline Random Forest, and metrics)



2. Dataset Preparation & Engineering (Phases 1 & 2)

Exploratory Data Analysis (EDA)

Instead of writing dozens of plotting functions, use ydata-profiling . It will automatically generate a comprehensive HTML report showing missing values, correlations, and distributions for every feature in the Kaggle dataset.

Preprocessing

- **Missing Values:** Use median imputation for continuous variables (e.g., DaySinceLastOrder) and mode imputation for categorical ones (e.g., PreferredLoginDevice).
- **Encoding:** EBMs handle categorical features natively. You *do not* need to one-hot encode them, which preserves their interpretability (the model will output rules for "Mobile" rather than "Device_Mobile_1").



- **Scaling:** EBM's are scale-invariant. You can skip Min-Max or Z-score normalization, keeping the raw, readable numbers for your dashboard.

Feature Engineering (Crucial for Urban Daraz)

When you move to Phase 2, raw logistics data won't work out of the box. You must engineer **RFM Metrics**:

- **Recency:** Days since the customer's last order.
- **Frequency:** Total number of successful orders in a given timeframe.
- **Monetary:** Average order value or total lifetime value.

3. Model Development (The EBM Core)

The `interpret` library makes training an EBM feel exactly like training a standard scikit-learn model.

Hyperparameter Tuning

EBMs are robust out of the box, but you should run a basic Grid Search to optimize:

- `interactions` : Set to 5 or 10. This allows the model to find combined rules (e.g., *High Tenure AND Low App Surfing* = Churn Risk).
- `learning_rate` : Default is 0.01; test 0.05 for faster convergence if the dataset is large.
- `max_bins` : Keep around 256 for optimal binning of continuous features.

The Baseline Comparison

To fulfill the comparative analysis requirement, train a standard `RandomForestClassifier` from `scikit-learn` . You will compare the EBM against this to prove that you didn't sacrifice accuracy to achieve interpretability.

4. Explainability & BI Export (The Bridge)

This is the most critical technical script (`export_bi.py`). Power BI cannot natively run EBM models dynamically. You must run the model in Python and export the results as clean CSV files for the dashboard.

1. **Global Explanations (The "Why"):** Export the `terms_scores_` to a CSV. This tells Power BI the overall importance of each feature across the entire business.
2. **Local Explanations (The "Who"):** Score your active customer base. Create a CSV containing:
 - `CustomerID`
 - `Churn_Probability` (e.g., 85% risk)
 - `Top_Risk_Factor_1` (e.g., `DaySinceLastOrder > 30`)

- `Top_Risk_Factor_2` (e.g., `Satisfaction_Score < 3`)

5. Business Intelligence Architecture (Power BI)

Lean entirely into your graphic design expertise here. The dashboard is the final DSR artifact and will be heavily scrutinized.

- **Figma Integration:** Do not use Power BI's default shapes and containers. Design a clean, minimalistic, modern background in Figma. Export it as an SVG and import it into Power BI as the canvas background.
- **Data Model:** Import your CSVs. Set up a simple Star Schema linking your `Customer_Profiles` table to the `EBM_Risk_Scores` table.
- **Required Visuals:**
 - *High-Risk Customer Table:* Filtered to show customers with a >70% churn probability, displaying their top risk factors.
 - *Global Feature Waterfall:* Showing how factors like delivery time or cashback amounts drive overall churn.
 - *Segment Slicers:* Allow the user to filter by City Tier or Device.

6. Evaluation Framework

To satisfy both the Machine Learning and Design Science criteria:

1. **Technical Validation (Python):** * Generate a classification report: Accuracy, Precision, Recall, F1-Score.
 - Plot the ROC-AUC curve comparing the EBM and the Random Forest.
2. **Practical Validation (Dashboard):** * Implement the System Usability Scale (SUS). Create a 10-question Google Form.
 - Have 3-5 colleagues or interns run through a scenario (e.g., "Identify the top 10 highest-risk customers in Tier 1 cities and list their main churn reason"). Have them rate the dashboard's ease of use.

This architecture minimizes complex coding and maximizes the impact of the final visual artifact.

Before we write the first line of code in `preprocess.py`, do you want to finalize the specific list of RFM and behavioral features we can realistically extract from the Urban Daraz systems for Phase 2?

the data set i going to use "

<https://www.kaggle.com/datasets/samuelsemaya/e-commerce-customer-churn>"

Gemini 3.1 Pro Extended

python

```
import pandas as pd
df = pd.read_csv('data_ecommerce_customer_churn.csv')

info = []
info.append("=== HEAD ===")
info.append(str(df.head()))
info.append("\n=== INFO ===")

# Capture df.info()
import io
buffer = io.StringIO()
df.info(buf=buffer)
info.append(buffer.getvalue())

info.append("\n=== TARGET VARIABLE (Churn) ===")
if 'Churn' in df.columns:
    info.append(str(df['Churn'].value_counts(normalize=True)))
else:
    info.append("No 'Churn' column found.")

print('\n'.join(info))
```

text

```
=== HEAD ===
   Tenure  WarehouseToHome  NumberOfDeviceRegistered  PreferredOrderCat  SatisfactionScore
MaritalStatus  NumberOfAddress  Complain  DaySinceLastOrder  CashbackAmount  Churn
0    15.0             29.0                4  Laptop & Accessory                3
Single                2          0              7.0             143.32          0
1     7.0             25.0                4           Mobile                1
Married              2          0              7.0             129.29          0
```

2	27.0	13.0	3	Laptop & Accessory	1
Married	5	0	7.0	168.54	0
3	20.0	25.0	4	Fashion	3
Divorced	7	0	NaN	230.27	0
4	30.0	15.0	4	Others	4
Single	8	0	8.0	322.17	0

=== INFO ===

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 3941 entries, 0 to 3940

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Tenure	3747 non-null	float64
1	WarehouseToHome	3772 non-null	float64
2	NumberOfDeviceRegistered	3941 non-null	int64
3	PreferedOrderCat	3941 non-null	object
4	SatisfactionScore	3941 non-null	int64
5	MaritalStatus	3941 non-null	object
6	NumberOfAddress	3941 non-null	int64
7	Complain	3941 non-null	int64
8	DaySinceLastOrder	3728 non-null	float64
9	CashbackAmount	3941 non-null	float64
10	Churn	3941 non-null	int64

dtypes: float64(4), int64(5), object(2)

memory usage: 338.8+ KB

=== TARGET VARIABLE (Churn) ===

Churn

0 0.828977

1 0.171023

Name: proportion, dtype: float64

Excellent. I have securely loaded the `data_ecommerce_customer_churn.csv` dataset you provided and run an initial background diagnostic.

As your Senior ML Engineer, I love what I am seeing, but I have also spotted a critical discrepancy between this dataset and your Project Master Document that we need to address before building the EBM.

Here is the diagnostic report of the Phase 1 Data you just provided:



Dataset Diagnostic Report

- **Total Records:** 3,941 customers

- **Target Variable (Churn):** 17.1% Churn Rate vs. 82.9% Retention Rate. This is a very realistic class imbalance for e-commerce.
- **Missing Values Detected:**
 - Tenure : ~194 missing values
 - WarehouseToHome : ~169 missing values
 - DaySinceLastOrder : ~213 missing values

⚠ Critical Supervisor Observation

Your thesis proposal listed 16 features for this Kaggle dataset (including App_surfing_inHrs , Payment_Method , CityTier , etc.). However, **this specific CSV only contains 10 features + the target variable.**

- **Current Features:** Tenure , WarehouseToHome , NumberOfDeviceRegistered , PreferredOrderCat , SatisfactionScore , MaritalStatus , NumberOfAddress , Complain , DaySinceLastOrder , CashbackAmount .

Why this is actually a good thing: A smaller, more concise feature set is *much* better for your BI dashboard. Trying to visualize 16 different feature impacts on Power BI gets cluttered. 10 features hit the "sweet spot" for UI design and cognitive load for non-technical SME managers. We will update the documentation to reflect these exact 10 features.

🔧 Next Step: Data Preprocessing (src/preprocess.py)

Since we dropped the Deep Learning component, we don't need complicated data augmentation. The Explainable Boosting Machine (EBM) is brilliant because it handles categorical text (like "Laptop & Accessory") natively. It doesn't need One-Hot Encoding, which keeps our Power BI outputs readable!

Here is the exact preprocessing script we need to run to prepare this data for the model.

python

```
import pandas as pd
from sklearn.model_selection import train_test_split

def prepare_data(file_path):
    print("Loading raw data...")
    df = pd.read_csv(file_path)

    print("Handling missing values...")
    # For EBMs, we use the Median for continuous variables to prevent outlier skewing
    df['Tenure'] = df['Tenure'].fillna(df['Tenure'].median())
    df['WarehouseToHome'] = df['WarehouseToHome'].fillna(df['WarehouseToHome'].median())
    df['DaySinceLastOrder'] = df['DaySinceLastOrder'].fillna(df['DaySinceLastOrder'].median())
```



```

# Separate features (X) and target (y)
X = df.drop(columns=['Churn'])
y = df['Churn']

print("Splitting into Train/Test sets (80/20)...")
# Stratify ensures the 17% churn rate is equally distributed in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f"Preprocessing Complete! Train shape: {X_train.shape}, Test shape: {X_test.shape}")
return X_train, X_test, y_train, y_test

# You would run this to get your pristine modeling data
# X_train, X_test, y_train, y_test = prepare_data('data_ecommerce_customer_churn.csv')

```

🎯 Your Action Items:

1. **Confirm the Feature List:** Are you okay with formally adjusting the thesis document to use these 10 specific features rather than the 16 originally proposed?
2. **Phase 2 Mapping:** Look at these 10 features (Tenure, Complaints, DaysSinceLastOrder, CashbackAmount, etc.). When we move to **Phase 2 (Urban Daraz data)**, which of these features do you think you can easily extract or calculate from your Shopify/Daraz store backend?

If you give me the green light, I will immediately draft the `train_ebm.py` script so we can train the model right now and generate the first CSVs for your Power BI dashboard!